Lecture Notes:
Model Diagnostics

CIC1205 - Machine Learning

Eduardo Bezerra

May 13, 2026

0 Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Diagnosis and the Bias–Variance Framework . . . 3
1.3 Overview of Diagnostic Tools . . . 4

2 Learning Curves **5**
2.1 Definition and Purpose . . . 5
2.2 How to Construct a Learning Curve . . . 6
2.3 Training vs. Validation Error: Key Observations . . . 7
2.4 Diagnosing High Bias from Learning Curves . . . 7
2.5 Diagnosing High Variance from Learning Curves . . . 8

3 Loss Curves **9**
3.1 Definition and Purpose . . . 9
3.2 Anatomy of a Loss Curve . . . 9
3.3 Diagnosing Underfitting from Loss Curves . . . 10
3.4 Diagnosing Overfitting from Loss Curves . . . 10
3.5 Other Patterns in Loss Curves . . . 12
3.5.1 Unrepresentative Training Set . . . 12
3.5.2 Unrepresentative or Small Validation Set . . . 12
3.6 Early Stopping as a Regularization Strategy . . . 12

4 Validation Curves **13**
4.1 Definition and Purpose . . . 13
4.2 Shape and Interpretation . . . 14
4.3 Relation to the Three Diagnostic Tools . . . 15

5 Error Analysis **15**
5.1 Definition and Purpose . . . 15
5.2 Confusion Matrix Analysis . . . 16
5.3 Per-Class Error Metrics . . . 16
5.4 Stratified Error Analysis for Regression with Skewed Targets . . . 17
5.5 Error Pattern Identification . . . 20
5.6 Feature Importance Analysis . . . 20
5.7 Visualizations for Error Analysis . . . 21
5.8 Cross-Validation for Error Stability . . . 21
5.9 Iterative Improvement Loop . . . 22
5.10 The Role of Domain Expertise . . . 22
5.11 Documentation of Error Analysis . . . 22

6 Data Leakage: A Special Diagnostic Concern **23**
6.1 Definition . . . 23
6.2 Types of Data Leakage . . . 23
6.2.1 Target Leakage . . . 23
6.2.2 Preprocessing Leakage . . . 24
6.2.3 Temporal Leakage . . . 25
6.2.4 Group Leakage . . . 25

6.3 Detecting Leakage 25

7 Final Remarks **26**
7.1 Connecting Diagnostics to Courses of Action 26
7.2 The Diagnostic Workflow 26
7.3 A Unified View of Model Improvement 27

1 Introduction

1.1 Motivation

Suppose you have built a first version of a machine learning model, and its predictive performance is unacceptable. What should you do next? Several courses of action seem reasonable:

- Collect more training examples.
- Train the algorithm for more iterations (more epochs).
- Increase or decrease the complexity of the model.
- Expand or reduce the feature set.
- Increase or decrease the regularization strength.

Each of these interventions takes time and resources. Without guidance, a practitioner may invest weeks in the wrong direction. For for example, gathering more data when the real problem is that the model is too simple to capture the underlying pattern.

Model Diagnostics

Model diagnostics is the discipline of using principled techniques to gain insight into the strengths and weaknesses of a trained machine learning model, in order to determine which improvements are most likely to be beneficial.

The central goal of model diagnostics is to answer two questions before committing to any course of action: *What is going wrong with the model?* and *Why is it going wrong?* A proper diagnosis rules out unpromising paths and focuses effort where it is most likely to help.

1.2 Diagnosis and the Bias–Variance Framework

The most useful conceptual lens for model diagnostics is the **bias–variance decomposition**. For squared-error regression, the expected prediction error at a fixed input $\mathbf{x}$ can be decomposed into three terms:

$$\mathbb{E}\big[(Y - \hat{f}(\mathbf{x}))^2\big] = \mathrm{Bias}^2\big[\hat{f}(\mathbf{x})\big] + \mathrm{Var}\big[\hat{f}(\mathbf{x})\big] + \sigma_\varepsilon^2,$$

where the expectation is taken over possible training sets drawn from the same distribution, and σ_ε^2 is the irreducible noise in the target variable.

Scope of the decomposition

The exact form above applies to regression with squared error loss. For classification problems and other loss functions, the same decomposition does not hold in this simple form. Nevertheless, the terms *bias* and *variance* remain useful as a diagnostic vocabulary: a model can still suffer from systematic error (high bias) or from excessive sensitivity to the training sample (high variance), and the remedies described below apply regardless of the task type.

- **Bias** measures how far the average prediction of the model deviates from the true value. A high-bias model is systematically wrong: it is too simple to capture the real structure in the data. This is called *underfitting*.
- **Variance** measures how much the model's predictions fluctuate across different training sets drawn from the same distribution. A high-variance model is too sensitive to the specifics of the training data; it has essentially memorized noise. This is called *overfitting*.
- **Irreducible noise** is the inherent randomness in the target variable that no model can remove.

Table 1 summarizes how the two failure modes manifest in practice, and what remedies are appropriate for each.

	High Bias (underfitting)	**High Variance (overfitting)**
Training error	High	Low
Validation error	High	High
Gap (val $\rightleftharpoons$ train)	Small	Large
Remedies	Use a more complex model	Collect more training data
	Add more/better features	Use stronger regularization
	Decrease regularization	Reduce model complexity
	Train for more iterations	Reduce or select features

Table 1: Symptoms and remedies for high bias and high variance.

Worked example: diagnosing from errors

Suppose `Model A` has training error 0.35 and validation error 0.38. Both errors are high and close to each other, suggesting *high bias*. The model is likely too simple; using a more complex model or adding better features may help.
Suppose `Model B` has training error 0.03 and validation error 0.32. The training error is low but the validation error is much higher, suggesting *high variance*. The model has overfit the training data; regularization, feature selection, or collecting more data may help.

The diagnostic tools described in the following sections (learning curves, loss curves, and validation curves) are precisely designed to reveal whether a model is suffering from high bias, high variance, or neither.

1.3 Overview of Diagnostic Tools

These notes cover four complementary diagnostics:

1. **Learning curves:** performance vs. training set size. Purpose: diagnose data sufficiency, underfitting, and overfitting.

2. **Loss curves:** training and validation loss vs. epoch. Purpose: diagnose optimization dynamics and overfitting during iterative training.
3. **Validation curves:** performance vs. a single hyperparameter value. Purpose: diagnose the effect of model complexity on generalization.
4. **Error analysis:** qualitative and quantitative inspection of misclassified or poorly predicted examples. Purpose: diagnose systematic error patterns.

Each tool addresses a different question and is most useful in a different context. A thorough diagnostic workflow typically employs all four.

Entry Ticket 1

Before reading further, consider a situation in which your model achieves 95% accuracy on the training set but only 68% on the validation set.

1. Is this model likely suffering from high bias or high variance? Justify your answer.
2. List two concrete actions that would most likely improve generalization.
3. Which diagnostic tool would you use first to investigate the problem?

2 Learning Curves

2.1 Definition and Purpose

Learning Curve

A **learning curve** is a plot of a performance metric (e.g., accuracy or mean squared error) as a function of the number of training examples used, for a fixed model and fixed hyperparameter configuration.

As the training set grows, we generally expect the model's generalization performance to improve: more data provides more signal, reducing the estimation variance of the learned parameters. At the same time, the training error typically increases slightly as more data are added, because the model can no longer perfectly memorize every example.

Learning curves are useful for:

- **Verifying that the algorithm is working correctly.** A correctly implemented algorithm will often show a validation error that tends to decrease as more data are added, although the curve is not guaranteed to be monotonic in finite samples. Random sampling, optimization noise, and fixed hyperparameter settings can all produce local fluctuations. The overall trend is therefore more informative than individual points.
- **Detecting underfitting and overfitting.** The relationship between the training and validation curves reveals whether the model has high bias or high variance (see Sections 2.4 and 2.5).
- **Evaluating data sufficiency.** If the validation curve is still declining at the rightmost point of the plot, collecting more data is likely to be beneficial.

- **Guiding decisions about model complexity.** The shape of the curves suggests whether to increase or decrease the capacity of the model.

2.2 How to Construct a Learning Curve

Constructing a learning curve requires training the model multiple times on progressively larger subsets of the training data:

1. Fix a model architecture and hyperparameter configuration.
2. Define a sequence of training set sizes: $n_1 < n_2 < \cdots < n_T = N$, where N is the total number of available training examples.
3. For each size n_t:
   (a) Randomly sample n_t examples from the training set (without replacement).
   (b) Train the model on the sampled subset.
   (c) Evaluate both the training error (on the subset used for training) and the validation error (on a held-out validation set that is *never* used for training).
4. Plot both curves against n_t.

Averaging over random subsamples

Because the error at each size n_t depends on which examples were randomly selected, it is good practice to repeat steps 3a–3c multiple times with different random seeds and report the average error at each size. This reduces the noise in the curve, especially for small n_t values.

Figure 1 shows the typical shape of a learning curve for a model that is well-specified relative to the data-generating process. As training size increases, the training error climbs slowly (the model can no longer memorize everything) while the validation error falls steadily, with the two curves converging toward a stable plateau.

Schematic learning curve: training and validation error

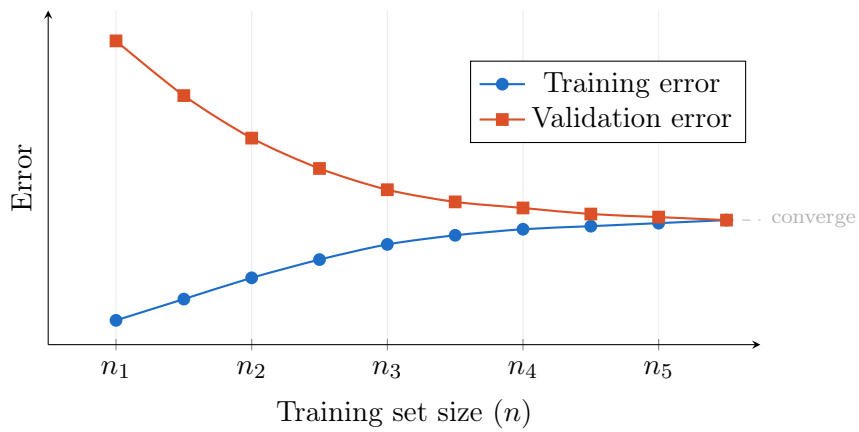

Figure 1: Schematic learning curve for a well-specified model. As training size increases, training error rises moderately while validation error decreases, with both curves converging toward a stable plateau.

2.3 Training vs. Validation Error: Key Observations

Two structural properties of learning curves are worth internalizing:

1. **Training error is usually lower than validation error.** The model has already seen the training examples during parameter fitting, so it tends to predict them better than examples it has never seen.
2. **Training error often increases with training set size.** With only a handful of training examples, a sufficiently flexible model can fit them almost perfectly, yielding a near-zero training error. As more examples are added, fitting all of them simultaneously becomes harder and the training error rises until it stabilizes near the irreducible noise level. This is a typical pattern for flexible models, not a strict mathematical guarantee for all algorithms and configurations.

2.4 Diagnosing High Bias from Learning Curves

A model suffering from *high bias* (underfitting) is too simple to capture the underlying pattern. Its learning curve has a distinctive signature:

- The training error converges to a *high* value; the model cannot fit even the training data well.
- The validation error also converges to a high value, close to the training error.
- The two curves reach a plateau quickly and remain near each other, with only a small gap between them.

Figure 2 illustrates this pattern. The key diagnostic features are that both curves plateau at a high error level, and that the gap between them is small.

High bias: both curves plateau at a high error, small gap between them

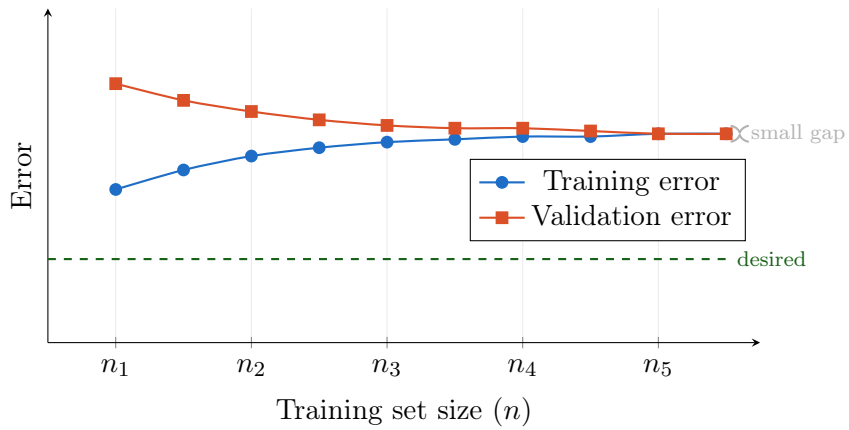

Figure 2: High-bias (underfitting) learning curve. Both training and validation errors converge to a high plateau with only a small gap between them, well above the desired performance level. Adding more data alone will not resolve this problem.

About the "desired performance" line

The desired performance line shown in the figures is not estimated from the learning curve itself. It represents an external requirement, such as a baseline to beat, a business constraint, or a minimum acceptable performance level for deployment. Defining this target in advance is good practice: it prevents the practitioner from stopping too early (when the model is still improvable) or too late (when the current performance already meets the need).

Key takeaway. When the learning curve has plateaued at a high error with a small train–validation gap, collecting more data is unlikely to produce a substantial improvement. The main bottleneck is model capacity or representation, not data quantity. The appropriate remedies are to use a more complex model, add more informative features, or reduce regularization.

2.5 Diagnosing High Variance from Learning Curves

A model suffering from *high variance* (overfitting) has too much capacity and has essentially memorized the training data, including its noise. Its learning curve signature is quite different from the high-bias case:

- The training error is *low*, because the model fits the training examples very well.
- The validation error is *much higher* than the training error, indicating poor generalization.
- There is a *large gap* between the two curves.
- As the training set grows, the validation error decreases and the two curves gradually approach each other, but the gap may remain substantial.

Figure 3 illustrates this pattern. The large vertical gap between the curves is the hallmark of high variance.

High variance: large gap between training and validation error

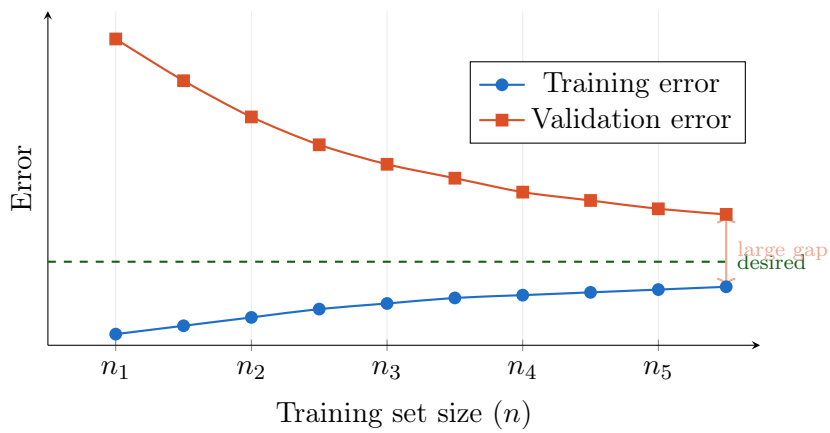

Figure 3: High-variance (overfitting) learning curve. The training error is low while the validation error is much higher, leaving a large gap. As more data are added, the curves approach each other and the validation error continues to decline.

Key takeaway. When a model suffers from high variance, collecting more training data is likely to help. As the training set grows, the model has less room to memorize noise and is forced to learn more general patterns. Other effective remedies include regularization, dropout (for neural networks), reducing model complexity, and feature selection.

Common mistake: misreading the gap

A large gap between training and validation error does not automatically mean that the model is too complex. It may also indicate that the training and validation sets come from different distributions, or that the validation set is too small and noisy. Always inspect the data split and its representativeness before concluding that the model needs to be simplified.

Entry Ticket 2

A decision tree classifier trained on a medical dataset of 500 patients achieves a training accuracy of 99% and a validation accuracy of 71%.

1. Sketch the expected shape of the learning curve for this model.
2. Would collecting 5 000 more patients likely close the gap? Justify your answer using the bias–variance framework.
3. Propose two alternative actions that would be more likely to improve validation accuracy without collecting more data.

3 Loss Curves

3.1 Definition and Purpose

Loss Curve

A **loss curve** (also called a *training curve* or *convergence curve*) is a plot of the value of the loss function as a function of training iteration or epoch, evaluated separately on the training set and on the validation set.

Loss curves are applicable to any learning algorithm that minimizes a loss function through an iterative optimization procedure (e.g., gradient descent). This includes linear regression, logistic regression, gradient boosting algorithms, and, most prominently, neural networks.

Unlike learning curves, which vary the amount of training data, loss curves vary the number of training steps. They reveal training dynamics: how the optimization evolves over time, whether the model converges, whether it converges too slowly or too quickly, and whether it begins to overfit at some point during training.

3.2 Anatomy of a Loss Curve

A typical loss curve plot shows two series:

- **Training loss:** computed on the training set at the end of each epoch (or at regular intervals during training).

- **Validation loss:** computed on a held-out validation set after each epoch, without using those examples for parameter updates.

In the ideal case, both curves decline smoothly and converge to stable low values with a small gap between them. Deviations from this pattern are diagnostic signals.

Figure 4 illustrates the loss curve of a well-fitted model.

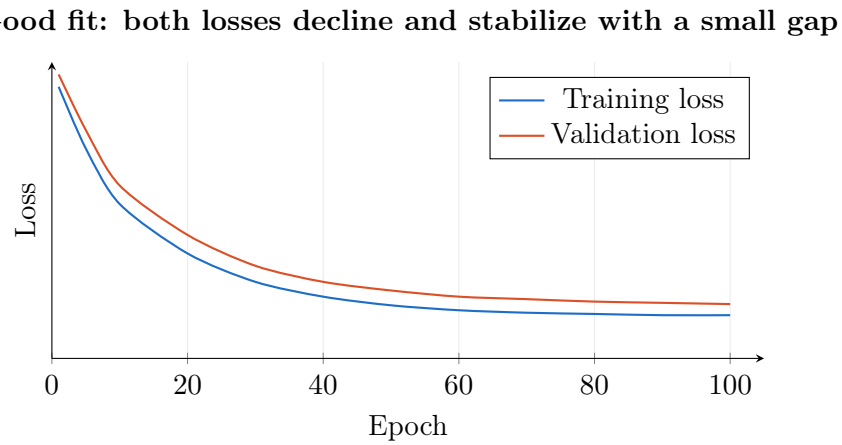

Figure 4: Good-fit loss curve. Training and validation losses decline to stable values with a small gap between them. This is the target behavior.

3.3 Diagnosing Underfitting from Loss Curves

An underfit model fails to capture the structure in the data. On a loss curve, this manifests in two characteristic patterns:

Pattern 1: Both losses remain high, with a small or moderate gap. Both the training loss and the validation loss are high throughout training, and the gap between them is small or moderate. The model is not fitting even the training data well, which is the hallmark of underfitting. Note that a *large* gap between training and validation loss is *not* a sign of underfitting; it is more characteristic of overfitting (see Section 3.4) or of a mismatch between training and validation distributions. Appropriate remedies include increasing model capacity, adding more informative features, reducing regularization, or improving the input representation.

Pattern 2: Curves still declining at the end. Both curves continue to decline at the last epoch of the plot, suggesting that training was stopped too early. The model has not yet converged. The appropriate remedy is to train for more epochs, potentially with a learning rate schedule or early-stopping callback.

Figure 5 illustrates both patterns.

3.4 Diagnosing Overfitting from Loss Curves

An overfit model memorizes the training data, including noise. On a loss curve, this produces a distinctive divergence pattern:

Underfitting: both losses high; curves may still be declining

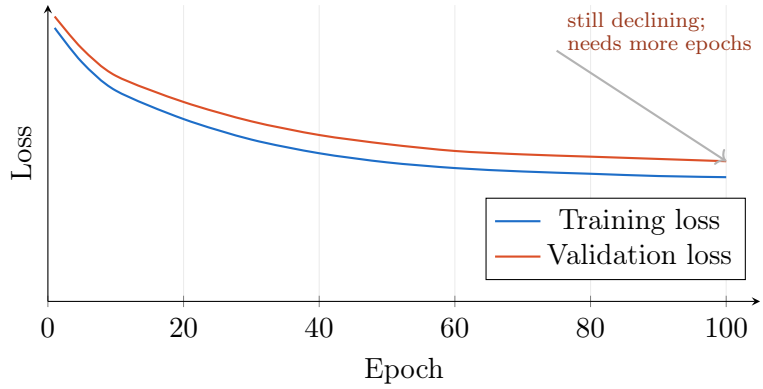

Figure 5: Underfitting loss curve. Both losses remain high throughout training, with only a small gap between them. The curves may still be declining at the end of training (suggesting more epochs are needed), or may have plateaued at an unsatisfactorily high level (suggesting the model is too simple).

- Both the training and validation losses initially decrease together.
- After a certain epoch, the training loss continues to decrease while the validation loss starts to increase.
- The point at which the validation loss begins to rise is the onset of overfitting.

This divergence defines the concept of *early stopping*: the best candidate for the number of training epochs is identified at the point where the validation loss reaches its minimum, before it begins to climb.

Figure 6 illustrates this pattern.

Overfitting: validation loss rises after an optimal point

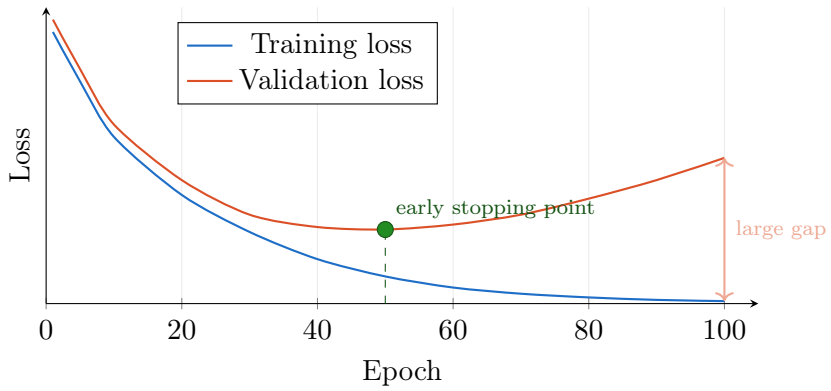

Figure 6: Overfitting loss curve. Training loss decreases monotonically while validation loss reaches a minimum and then rises. The green dot marks the early-stopping point, i.e., the epoch with the best observed validation loss.

3.5 Other Patterns in Loss Curves

Beyond the canonical underfitting and overfitting patterns, two additional situations are commonly encountered in practice.

3.5.1 Unrepresentative Training Set

When the training set is not representative of the broader data distribution (for example, if training examples were collected under different conditions than test examples) both the training loss and the validation loss may converge to stable values, but a *large and persistent gap* remains between them even after training is complete. This differs from the overfitting pattern in that the validation loss does not rise; it simply stabilizes at a value much higher than the training loss.

The appropriate diagnosis is that the training distribution differs from the validation distribution. Remedies include resampling the training data to better match the target distribution, applying domain adaptation techniques, or collecting new training data that better reflects the deployment conditions.

3.5.2 Unrepresentative or Small Validation Set

When the validation set is very small or not representative of the full distribution, the validation loss curve may exhibit *high noise*: large fluctuations from epoch to epoch, even when the training loss is smooth and well-behaved. This noisy behavior is not a property of the model but of the evaluation procedure: the validation metric is estimated from too few examples and therefore has high variance.

The remedy is to increase the size of the validation set, use k-fold cross-validation instead of a single holdout split, or apply exponential smoothing to the plotted curve for visual clarity.

3.6 Early Stopping as a Regularization Strategy

The loss curve analysis motivates a practical and widely used regularization technique known as **early stopping**:

1. Monitor the validation loss at the end of each training epoch.
2. Keep a record of the model parameters at the epoch with the lowest validation loss observed so far.
3. If the validation loss does not improve for a specified number of consecutive epochs (the *patience* parameter), halt training.
4. Restore the model parameters saved at the best validation epoch.

Early stopping implicitly penalizes over-training and is particularly effective for neural networks, where the model can in principle achieve arbitrarily low training loss given enough epochs.

Early Stopping Rule

The selected stopping point is determined by the validation loss curve, not by the training loss curve. Training should stop when the validation loss reaches its minimum, regardless of what the training loss is doing at that point. In practice, a patience parameter is used to avoid reacting to random fluctuations, and the model parameters from the best observed epoch are restored at the end of training. This epoch is an estimate of the best stopping point, not a guarantee of the globally optimal model.

Entry Ticket 3

A practitioner trains a neural network for 200 epochs on a tabular dataset. At epoch 80, the validation loss reaches 0.45. From epoch 80 onward, the training loss continues to drop from 0.30 to 0.05, while the validation loss climbs from 0.45 to 0.90 by epoch 200.

1. What type of behavior does this loss curve describe?
2. At which epoch should training have been stopped?
3. If the practitioner evaluates the model only after epoch 200, will this result reflect the best model encountered during training? Why or why not?

4 Validation Curves

4.1 Definition and Purpose

Validation Curve

A **validation curve** is a plot of a performance metric (training error and validation error) as a function of a single hyperparameter value, with all other hyperparameters held fixed.

While learning curves vary the *amount of data* and loss curves vary the number of *training iterations*, validation curves vary the *model complexity* as controlled by a specific hyperparameter (e.g., the maximum depth of a decision tree, the regularization constant C in an SVM, the number of layers in a neural network, or the number of neighbors k in a k-NN classifier).

Validation curves are used to:

- Identify the range of a hyperparameter in which the model underfits or overfits.
- Find the value at which the validation error is minimized (the optimal complexity point).
- Understand the sensitivity of the model to changes in a particular hyperparameter.

Note that validation curves and hyperparameter tuning are closely related: the former is a *diagnostic* tool (it reveals the bias–variance behavior of the model across a range of values), while the latter is an *optimization* task (it searches the hyperparameter space for the best configuration). A validation curve visualizes what tuning is trying to find.

4.2 Shape and Interpretation

The canonical shape of a validation curve is a `U`-shape for the validation error when the x-axis is ordered from low to high model complexity. As model complexity increases:

- At low complexity (left side of the plot): both training and validation errors are high. The model underfits.
- At moderate complexity (center of the plot): the validation error reaches its minimum, the bias–variance trade-off is well balanced.
- At high complexity (right side of the plot): the training error continues to fall while the validation error rises. The model overfits.

Meanwhile, the training error decreases monotonically (or near-monotonically) as complexity increases: a more powerful model can always fit the training data at least as well as a simpler one.

Hyperparameter direction matters

The U-shape is easiest to interpret when the x-axis is ordered from low to high effective model complexity. However, some hyperparameters *increase* complexity as their value increases (e.g., tree depth, number of layers), while others *decrease* complexity as their value increases (e.g., the number of neighbors k in k-NN, or the regularization strength λ in Ridge regression). In the latter case, the validation curve will appear as an inverted U-shape from left to right. The curve should therefore always be interpreted in terms of effective model complexity, not simply in terms of the numerical value of the hyperparameter.

Figure 7 shows the validation curve for the `max_depth` hyperparameter of a decision tree regressor, an example where increasing the hyperparameter value directly increases model complexity.

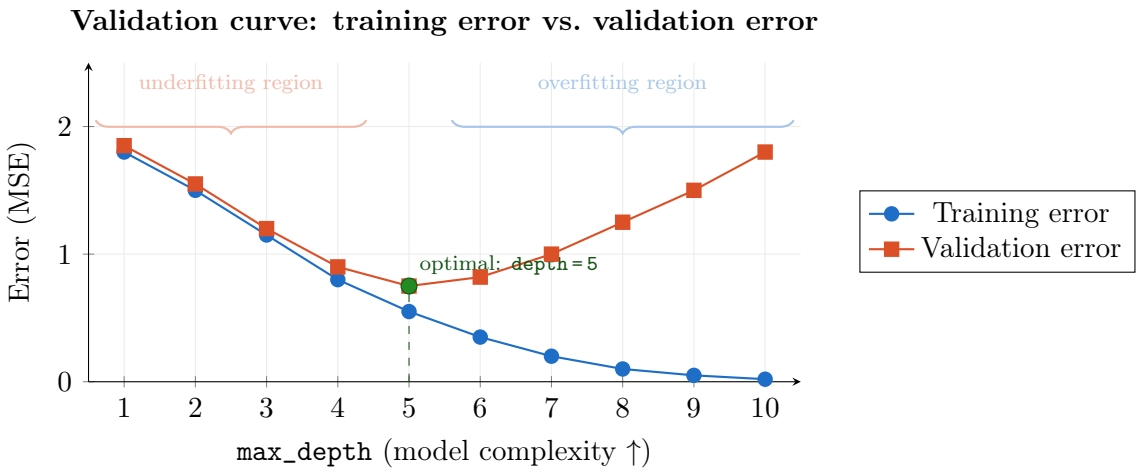

Figure 7: Validation curve for the `max_depth` hyperparameter of a decision tree. The training error decreases monotonically while the validation error follows a U-shape, reaching its minimum at `max_depth = 5`. This is the optimal operating point.

4.3 Relation to the Three Diagnostic Tools

Before consulting the comparison table, it is worth highlighting a common source of confusion: all three curve types plot training and validation performance, yet they vary fundamentally different quantities. In a *learning curve*, the question is what happens when the amount of training data changes. In a *loss curve*, the question is what happens as training progresses through epochs. In a *validation curve*, the question is what happens as model complexity changes. As a result, the same visual pattern can carry different interpretations depending on which axis is being varied.

Table 2 summarizes the three tools.

Curve type	**x-axis**	**Primary question**	**Applicable to**
Learning curve	Training set size	Is more data needed?	All algorithms
Loss curve	Epoch / iteration	Is training converging well?	Iterative optimizers
Validation curve	Hyperparameter value	What is the optimal complexity?	All algorithms

Table 2: Comparison of the three main diagnostic curve types.

In practice, the three tools are used together. A validation curve identifies the best hyperparameter setting; then, at that setting, a learning curve checks whether more data would further improve generalization; and if the algorithm is iterative, a loss curve verifies that optimization is stable.

Entry Ticket 4

You train an SVM classifier with a radial basis function (RBF) kernel and produce a validation curve by varying the regularization parameter C over the range $\{0.001, 0.01, 0.1, 1, 10, 100\}$.

1. For very small C values, do you expect the training and validation errors to be high or low? Why?
2. For very large C values, do you expect a large or small gap between training and validation errors? Why?
3. How would you use this curve to decide which value of C to use?

5 Error Analysis

5.1 Definition and Purpose

The three curve-based diagnostics described in the previous sections operate at the level of aggregate statistics: they tell us *how much* a model errs, but not *which* examples are difficult or *why* the model fails on them. Error analysis fills this gap.

Error Analysis

Error analysis is the process of manually or systematically examining the examples on which a model makes mistakes, with the goal of identifying patterns or systematic tendencies that explain the errors and suggest targeted improvements.

Error analysis typically involves two steps: (1) collecting a sample of misclassified or poorly predicted examples, and (2) analyzing these examples to identify patterns or trends. It is an iterative activity: the insights gained from one round of analysis lead to hypotheses, which are tested by modifying the model or data, and the results are analyzed again.

5.2 Confusion Matrix Analysis

For classification problems, the **confusion matrix** is the primary starting point for error analysis. A confusion matrix is a $K \times K$ table where entry (i, j) counts the number of examples belonging to class i that the model predicted as class j. The diagonal entries are correct predictions; all off-diagonal entries are errors.

Predicted class

	Class 1	**Class 2**	**Class 3**
Class 1	**85**	**5**	**2**
Class 2	**40**	**12**	**8**
Class 3	**3**	**5**	**70**

large error: 40 examples of class 2 predicted as class 1

Figure 8: Example confusion matrix for a three-class problem. The diagonal cells (green) represent correct predictions. The highlighted cell (red) reveals that class 2 is heavily misclassified as class 1, a pattern that warrants investigation.

Figure 8 shows an example with three classes where class 2 is systematically confused with class 1. This pattern immediately suggests avenues for investigation: Are classes 1 and 2 visually or statistically similar? Is the training set imbalanced, with fewer class 2 examples? Are there features that better discriminate between the two classes?

5.3 Per-Class Error Metrics

Beyond the overall confusion matrix, computing **per-class metrics** provides finer-grained insight into model performance. For each class c, the following metrics can be calculated:

- **Precision** for class c: of all examples predicted as class c, what fraction truly belongs to class c?

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}$$

- **Recall** (sensitivity) for class c: of all true members of class c, what fraction did the model correctly identify?
$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$$

- **F1-score** for class c: harmonic mean of precision and recall, providing a single summary metric:
$$F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

A class with notably low recall is one where the model frequently misses true positives; a class with low precision is one where the model generates many false alarms. Identifying which classes have poor metrics focuses subsequent investigation.

5.4 Stratified Error Analysis for Regression with Skewed Targets

In regression problems, aggregate metrics such as MAE, RMSE, or MSE computed over the entire test set can give a misleading picture of model performance when the distribution of the target variable is heavily skewed or has a long tail. This situation is common in environmental and scientific applications (e.g., daily precipitation, river discharge, wind speed, energy demand peaks), but arises in any domain where extreme values are rare yet consequential.

Aggregate metrics can hide failures on rare but critical values

When the target distribution is heavily skewed, a low aggregate error does not guarantee good performance across the full range of target values. The bulk of the distribution dominates the aggregate metric, potentially concealing systematic errors on rare extreme values that are often the most consequential for decision-making. A model that predicts low-intensity events well but fails consistently on extremes can still report an excellent overall MAE or RMSE.

The intuition is straightforward. Suppose 95% of test days have precipitation below 5 mm and only 5% have precipitation above 50 mm. The overall MAE is essentially a weighted average dominated by the low-precipitation regime. A model that systematically underestimates extreme events will contribute very little error to that average, because the extreme days are so few.

The stratification approach. The remedy is to partition the test set into **strata** defined by intervals of the observed target value, and to compute error metrics separately within each stratum. For daily precipitation, a domain-informed partition might follow meteorological conventions:

For each stratum $S_k = \{i : y_i \in I_k\}$, three per-stratum metrics are particularly informative:

- **MAE per stratum:**
$$\text{MAE}_k = \frac{1}{|S_k|} \sum_{i \in S_k} |\hat{y}_i - y_i|$$
measures average absolute error within the regime, on the same scale as the target.

Stratum	Range (mm/day)	Common label	Typical frequency
1	$y = 0$	No rain	High
2	$0 < y \leq 1$	Trace	Moderate
3	$1 < y \leq 10$	Light rain	Moderate
4	$10 < y \leq 50$	Moderate/heavy	Low
5	$y > 50$	Extreme	Very low

Table 3: Example stratification scheme for daily precipitation. Strata should be defined using domain knowledge; quantile-based splits are an alternative when no natural boundaries exist.

- **RMSE per stratum:**

$$\mathrm{RMSE}_k = \sqrt{\frac{1}{|S_k|} \sum_{i \in S_k} (\hat{y}_i - y_i)^2}$$

  is more sensitive to large individual errors within the stratum and is preferable when the cost of a single severe miss is high.

- **Mean Bias Error (MBE) per stratum:**

$$\mathrm{MBE}_k = \frac{1}{|S_k|} \sum_{i \in S_k} (\hat{y}_i - y_i)$$

  detects *systematic* under- or overestimation. A negative MBE in the highest stratum is a common and practically important failure mode: the model consistently underestimates extreme events, a pattern invisible in the aggregate metrics.

Why MBE matters for extremes

MAE and RMSE measure the magnitude of the error but not its direction. A model that alternates between overestimating and underestimating extreme events can have a low MAE while its MBE is near zero, hiding the fact that it is poorly calibrated. Conversely, a strongly negative MBE in the top stratum reveals a systematic regression-to-the-mean behavior: the model has learned that extreme values are rare and "plays it safe" by predicting conservatively. This is a bias the aggregate error will not expose.

Recommended visualizations. Two complementary plots make stratified errors easy to communicate:

- **Conditional residual plot.** Plot the residual $\hat{y}_i - y_i$ on the y-axis against the observed value y_i on the x-axis, with a loess or binned smoothing curve overlaid. A well-calibrated model shows residuals centered on zero across the full range. The typical failure pattern for skewed targets is a downward slope at high values (underestimation of extremes) or a funnel shape (variance growing with the target magnitude).

- **Binned metrics bar chart.** A grouped bar chart with strata on the x-axis showing both the per-stratum metric (e.g., MAE) and the relative frequency of each stratum. This makes it immediately apparent where the model fails and how rare those cases are, which is essential for communicating the practical significance of the errors to decision-makers.

Connecting diagnostics to remedies. Table 4 maps the most common per-stratum patterns to their likely causes and suggested actions.

Pattern observed	Likely cause	Suggested action
MBE strongly negative in top strata	Systematic underestimation of extremes (regression to the mean)	Use an asymmetric loss function (e.g., quantile loss, Huber loss); apply post-hoc calibration
MAE in top strata much larger than in lower strata	Model under-trained on rare extreme regime	Oversample or up-weight extreme examples; engineer features specific to high-intensity conditions
Very few test examples in the highest stratum	Unreliable per-stratum estimate due to small sample	Report confidence intervals for that stratum; collect more data in the extreme regime before drawing conclusions
Good aggregate MAE but large MBE in several strata	Errors cancel across strata, masking systematic bias	Never rely solely on aggregate metrics; always inspect MBE per stratum before declaring the model acceptable

Table 4: Common per-stratum diagnostic patterns in regression with skewed targets, and their associated remedies.

Relation to classification diagnostics. The logic of stratified error analysis in regression is the same as per-class metrics in classification (Section 5): in both cases, a single aggregate metric conceals heterogeneous performance across subgroups, and the appropriate response is to disaggregate. In classification, the subgroups are defined by the discrete class label; here, they are defined by intervals of the continuous target. The analogy between recall on a rare class and MAE on a rare target regime is direct.

Entry Ticket 5

A regression model for daily precipitation achieves an overall MAE of 1.2 mm on the test set.

1. Days with precipitation above 30 mm correspond to only 3% of the test set.

Should you trust the overall MAE as a complete summary of the model's performance? Why or why not?

2. Propose a stratification scheme for this problem and name two per-stratum metrics you would compute.
3. If the per-stratum MBE is strongly negative for the highest-intensity stratum, what does this indicate about the model's behavior on extreme events, and what remedies would you consider?

5.5 Error Pattern Identification

After identifying the *which* (which classes or examples are misclassified), the next step is to understand the *why*. Errors in machine learning models tend to cluster along identifiable dimensions:

- **Specific types of input.** For example, a speech recognition model may fail more often on examples recorded in noisy environments. An image classifier may struggle with blurry or low-resolution images.
- **Certain classes or label values.** Some classes may be inherently more difficult to distinguish, or may be underrepresented in the training set.
- **Particular feature combinations or ranges.** The model may be well-calibrated for typical feature values but fail on extreme or rare combinations. Visualizing errors in the feature space can reveal such boundaries.

A systematic approach is to build a small **error taxonomy**: examine 50–200 misclassified examples, manually assign each to a category (e.g., "blurry image," "unusual lighting," "occlusion"), and count the frequency of each category. This reveals where the model's effort should be focused.

Mini-case: fraud detection

A fraud classifier achieves 96% overall accuracy, but recall for the fraud class is only 30% (i.e., 70% of actual fraud cases are missed). High accuracy is misleading here because fraud is rare: the model can reach high accuracy simply by predicting "not fraud" almost all the time.

A first error analysis step is to collect the false negatives (i.e., fraudulent transactions that the model labeled as non-fraudulent) and examine them manually. After reviewing 100 such cases, the team might find that 45 involve small repeated transactions (potential structuring), 30 involve purchases at new or unfamiliar merchants, and 25 involve international transactions. This taxonomy suggests targeted fixes: engineer frequency-based features to capture structuring behavior, add merchant history features, and rebalance the training sample for international transactions. This illustrates the connection between the confusion matrix, per-class recall, and an actionable error taxonomy.

5.6 Feature Importance Analysis

Analyzing the importance of individual features in the model's predictions can reveal whether certain features are driving errors, or whether irrelevant features are introducing

noise that degrades performance.

Methods for feature importance assessment include:

- **Built-in importance scores:** tree-based models (random forests, gradient boosting) provide impurity-based feature importance scores as a by-product of training. These scores are convenient, but should be interpreted with care: they may be biased toward continuous or high-cardinality features, and they do not imply that a feature has a causal effect on the target.
- **Permutation importance (probe method):** for any model type, randomly shuffle the values of a single feature in the validation set and measure the resulting increase in validation error. A large increase indicates that the feature is important; no change indicates the feature is not being used. Unlike impurity-based scores, permutation importance is evaluated on held-out data and is therefore less susceptible to overfitting artifacts.
- **SHAP values:** a game-theoretic approach that assigns to each feature a contribution value for each individual prediction, enabling both global and instance-level feature importance analysis.

Feature importance analysis serves two purposes in error analysis: identifying which features *should* matter (by consulting domain knowledge) and verifying whether the model is actually using those features, or learning spurious correlations instead.

5.7 Visualizations for Error Analysis

Visual exploration of the data and the model's predictions can surface patterns that aggregate statistics miss. Useful visualization strategies include:

- **Scatter plots** of two features, colored by whether the prediction was correct or incorrect. Decision boundaries can often be overlaid to understand where the model is making mistakes.
- **Histograms** of feature values, separated by correctly classified and misclassified examples. If the distributions differ systematically, certain feature ranges are associated with higher error rates.
- **Residual plots** (for regression): plotting the residual (predicted minus true value) against the predicted value or against individual features reveals systematic biases in the model's predictions.
- **Heat maps** of confusion matrices or error rates across combinations of categorical features, useful for identifying specific subgroups where the model underperforms.

5.8 Cross-Validation for Error Stability

A single train–validation split may yield an error pattern that is specific to that particular partition. To assess whether identified error patterns are stable and reproducible, it is useful to perform k-fold cross-validation and examine whether the same classes or example types are consistently misclassified across folds. Stable errors are more likely to reflect genuine model weaknesses; errors that appear only in some folds are more likely to reflect data variability.

5.9 Iterative Improvement Loop

Error analysis is most effective as part of an iterative cycle:

1. Train a baseline model.
2. Evaluate on a validation set; compute error metrics.
3. Sample misclassified examples; categorize them; identify the most frequent error patterns.
4. Formulate a hypothesis about the root cause (e.g., "the model lacks a feature that distinguishes class A from class B").
5. Apply a targeted fix: collect more examples of the problematic type, engineer a new feature, adjust the class weights, or modify the model architecture.
6. Re-evaluate; compare metrics; confirm that the fix improved performance without degrading performance on other cases.
7. Return to step 3 until performance is satisfactory.

This loop is fundamentally different from blind hyperparameter search: each iteration is guided by specific diagnostic evidence rather than random exploration.

5.10 The Role of Domain Expertise

No diagnostic technique replaces domain knowledge. A practitioner who understands the data-generating process can often immediately recognize why a model is failing on a particular type of example, and can suggest targeted solutions that a purely statistical analysis would not reveal.

Domain experts should be consulted at two key points: *before* building the model (to identify which features are likely to matter and which data quality issues to anticipate) and *during error analysis* (to interpret the patterns found in misclassified examples and suggest plausible causes and remedies).

5.11 Documentation of Error Analysis

Results of error analysis should be documented in sufficient detail that they are reproducible and communicable. A useful error analysis report includes:

- The total number of misclassified examples examined.
- The taxonomy of error categories and the frequency of each.
- Representative examples from each error category (anonymized if necessary).
- The hypotheses formulated from the analysis.
- The actions taken in response, and their measured effect on performance.

Documentation also prevents teams from "re-discovering" the same error patterns across multiple iterations, and provides a record of what has already been tried.

Entry Ticket 6

A binary classifier for detecting fraudulent transactions achieves 96% accuracy on the test set, but the fraud detection team reports that it misses 70% of actual fraud cases.

1. Is the model's 96% accuracy a good overall measure of performance in this context? What metric would be more appropriate?
2. Sketch the type of confusion matrix you would expect for this model.
3. Propose a two-step error analysis plan to identify the most common types of fraud that the model is missing.

6 Data Leakage: A Special Diagnostic Concern

6.1 Definition

One of the most insidious failure modes in machine learning projects is *data leakage*, a situation in which information that would not be available at prediction time, or information from the validation or test set, is inadvertently used during training, preprocessing, feature engineering, model selection, or evaluation. Leakage causes the model to appear better on evaluation than it actually is in deployment.

Data Leakage

Data leakage occurs when information that would not be available at prediction time, or information from the validation or test set, influences the training process. Leakage leads to optimistically biased performance estimates that do not reflect real-world deployment performance.

A practical warning sign of data leakage is a model that achieves suspiciously high performance. For example, near-perfect accuracy on a difficult task, or a cross-validation score significantly higher than what is typically reported in the literature for the same problem. Any time evaluation metrics are surprisingly good, leakage should be actively investigated before trusting the results.

6.2 Types of Data Leakage

Data leakage manifests in several distinct forms, each with a different root cause and remedy. Table 5 provides an overview; the subsections that follow describe each type in detail.

6.2.1 Target Leakage

Target leakage occurs when a feature in the training data contains information about the target variable that would not be available at the time a prediction needs to be made in production.

Example. Consider a model trained to predict whether a hospital patient will be readmitted within 30 days of discharge. If the training dataset includes a feature called "number of follow-up visits in the first 30 days after discharge," this feature is derived from

Leakage type	**What goes wrong**	**Prevention**
Target leakage	A feature encodes information about the target or future outcome	Audit every feature for availability at prediction time
Preprocessing leakage	Transformations are fitted on the full dataset before splitting	Fit preprocessing only on training folds using a `Pipeline`
Temporal leakage	Future observations influence the model's learning for past time points	Use temporal splits or rolling/expanding windows
Group leakage	The same entity (patient, user) appears in both train and validation/test	Use group-aware splitting (`GroupKFold`)

Table 5: Summary of data leakage types, their root causes, and prevention strategies.

events that occur *after* the outcome is known. At prediction time (i.e., at the moment of discharge), this information is not yet available. Including it in training produces an artificially high training and test accuracy, but the model would be useless in deployment.

Remedy. Carefully audit each feature for temporal validity: the feature value must be computable from information available *at prediction time* only. For time-series data, this requires strict temporal splitting (see Section 6.2.3).

6.2.2 Preprocessing Leakage

Preprocessing leakage occurs when a data transformation step that learns from the data is applied to the entire dataset before the train–validation split, so that the validation set's statistical properties "leak" into the transformations applied during training.

Example. Suppose a practitioner computes the mean and standard deviation of each feature across all N examples and uses them to standardize the data. If this standardization is done before splitting, the training set's preprocessing has been informed by the validation and test examples. The learned mean and variance are not what would be obtained from the training set alone.

Remedy. All preprocessing steps that learn from the data (including feature scaling, imputation of missing values, PCA, feature selection, and target encoding) must be fitted *exclusively* on the training fold and then applied (without refitting) to the validation and test folds. In scikit-learn, this is achieved by encapsulating all preprocessing and the model in a `Pipeline` object and passing the pipeline to the cross-validation function.

Pipeline Rule

Never fit a preprocessing step on the full dataset before splitting. Always fit preprocessing inside each training fold and apply it to validation/test folds without refitting. Use a `Pipeline` to enforce this automatically.

6.2.3 Temporal Leakage

In time-series and temporal datasets, temporal leakage occurs when the training set contains observations from the future relative to the validation or test observations. A naive random split of a time-ordered dataset will mix past and future examples across all subsets, allowing the model to effectively look ahead.

Remedy. For temporal data, always use a temporal split: all training examples must have timestamps earlier than all validation examples, which in turn must be earlier than all test examples. Time-series cross-validation with an expanding or rolling window enforces this structure.

6.2.4 Group Leakage

Group leakage occurs when multiple observations belonging to the same entity (a patient, a user, a product) appear in both the training and validation sets. If the model can essentially memorize properties of individual entities, its validation performance will be overly optimistic because those same entities appear in training.

Example. A medical imaging classifier trained and validated on images from the same set of patients may learn patient-specific characteristics (e.g., image artifacts from a specific scanner) rather than the pathological patterns it is supposed to detect. Validation performance is inflated because the model has "seen" each patient during training.

Remedy. Use group-aware splitting: ensure that all examples from a given group (patient, user, etc.) appear in exactly one of training, validation, or test. In scikit-learn, `GroupKFold` implements this strategy.

6.3 Detecting Leakage

When performance is suspiciously high, the following checks can help detect leakage:

- **Temporal audit:** For each feature, verify that it could not have been derived from information available after the observation was collected.
- **Feature importance sanity check:** If a feature that should be uninformative (e.g., a record ID, a patient name, a timestamp) appears as a top predictor, it may be acting as a proxy for the target label.
- **Shuffle test:** Train the model with randomly shuffled target labels. If accuracy is above chance, a feature is directly encoding the target.
- **Ablation study:** Remove features one at a time and observe the effect on validation performance. A dramatic drop when a seemingly unimportant feature is removed suggests leakage.

- **Performance on a fresh hold-out:** If a model trained with leakage is evaluated on a completely independent dataset collected at a later date, performance will typically drop sharply. A large discrepancy between validation and real-world performance is a strong indicator of leakage.

7 Final Remarks

7.1 Connecting Diagnostics to Courses of Action

The central value of model diagnostics is that each diagnosis maps to a specific set of appropriate remedies. Table 6 summarizes the mapping developed throughout these notes.

Diagnostic finding	Likely cause	Recommended actions
Both train and val error high; small gap	High bias	Use more complex model; add features; decrease regularization; train longer
Train error low; val error much higher; large gap	High variance	Collect more data; regularize; reduce complexity; remove features
Val error declining at max data size	Insufficient data	Collect more training examples
Val loss rises after optimal epoch	Overfitting during training	Apply early stopping; increase regularization; reduce epochs
Loss still declining at end of training	Undertrained	Increase number of epochs; check learning rate
Specific class frequently misclassified	Class imbalance or missing features	Rebalance data; engineer discriminative features; adjust class weights
Suspiciously perfect performance	Data leakage	Audit features; check splits; use pipelines for preprocessing

Table 6: Summary of diagnostic findings and their associated remedies.

7.2 The Diagnostic Workflow

The following order of operations is recommended as a general diagnostic workflow for a model that underperforms:

1. **Check for obvious problems.** Are the training and validation sets properly constructed? Is there any suspected data leakage? If performance is suspiciously

good rather than bad, leakage is the first suspect.

2. **Examine the loss curve (if applicable).** Is the model converging? Did training stop too early? Is there a divergence between training and validation loss?
3. **Plot learning curves.** Is the model underfitting (both errors high, small gap) or overfitting (large gap)? Would more data help?
4. **Plot validation curves.** For the most important hyperparameter, is the model operating in the underfitting or overfitting region? What is the optimal complexity level?
5. **Conduct error analysis.** Examine the specific examples the model gets wrong. Are there systematic patterns? Which classes or input types are most problematic?
6. **Formulate and test hypotheses.** Based on the diagnostic findings, select a targeted improvement, implement it, and measure whether performance improved on the validation set.
7. **Iterate.** Repeat steps 2–6 until performance is satisfactory, then evaluate once on the held-out test set.

7.3 A Unified View of Model Improvement

The diagnostic tools described in these notes all share a common purpose: they help the practitioner answer *which* intervention to apply before committing to it. No diagnostic, however, replaces the need for careful experimental design. Improvements should always be validated on a held-out set, and the test set should remain sealed until all development decisions are complete.

The complementary relationship between model diagnostics (covered in these notes) and model selection (covered in previous notes) forms the foundation of a principled machine learning workflow. Diagnostics answer *what is wrong*; model selection answers *which configuration is best*. Together, they replace guesswork with evidence.

Key Takeaways

- **Learning curves** reveal whether a model suffers from high bias or high variance, and whether collecting more data is likely to help. High bias manifests as both errors high with a *small* gap; high variance manifests as a *large* gap between a low training error and a high validation error.
- **Loss curves** reveal whether an iteratively trained model is converging properly, and identify the candidate stopping point via early stopping. Underfitting shows both losses high; overfitting shows validation loss rising after an initial decline.
- **Validation curves** reveal the relationship between model complexity and generalization error. Interpret them in terms of effective model complexity, not just the numerical value of the hyperparameter.
- **Error analysis** reveals *which* examples are difficult and *why*, guiding targeted feature engineering and data collection.
- **Data leakage** can produce artificially high performance estimates and must be actively detected and prevented through proper pipeline design and

temporal/group-aware splitting. A large gap between validation and deployment performance is a strong warning sign.

- The correct diagnostic workflow is to apply these tools *before* committing to any improvement strategy, so that effort is directed where it is most likely to pay off.

7 References